Probabilistic Analysis

March 10, 2026

1 Probabilistic Analysis of Deterministic Algorithms

Probabilistic Analysis is the use of probability in the analysis of problems. Typically, we use probabilistic analysis to analyze the running time of an algorithm, but it can be used to analyze other quantities as well (that are dependent of some steps in the algorithm).

To perform a probabilistic analysis, we must know or make assumptions about the **distribution of the inputs**. Then we analyze our algorithm, computing an average-case running time, where we take the average over the distribution of the possible inputs. We are thus averaging the running time over all possible inputs. When reporting such a running time, we will refer to it as the **expected time** or **average-case running time**.

1.1 Deterministic vs. Randomized Algorithms

Normally, when we talk about the running time of an algorithm A, we mean the **worst-case** running time. This is the maximum, over all problems of a certain size, of the running time of that algorithm on that input:

$$T_{\text{worst-case}}(n) = \max_{|X|=n} T(X).$$

On very rare occasions, we may be interested in the **best-case** running time:

$$T_{\text{best-case}}(n) = \min_{|X|=n} T(X).$$

Sometimes the number of instances of size n with running time $T_{\text{worst-case}}(n)$ or close to $T_{\text{worst-case}}(n)$ is very small, and for such cases an average (expected) time is more appropriate.

The **average-case** running time is best defined by the **expected value**, over all inputs X of a certain size n, of the algorithm's running time for X:

$$exp - time(n) = T_{\text{average-case}}(n) = \mathop{\mathrm{E}}_{|X|=n}[T(X)] = \sum_{|X|=n} T(X) \cdot Pr\{X\}.$$

The problem is that we rarely, if ever, know the probability of getting a specific input X. To compute average-case running times we need to assume a particular probability distribution (for example, that every possible input is equally likely).

Let's recall that randomized algorithms base their behavior not only on the input they receive, but also on several random choices. The same randomized algorithm, on the same input, may perform different computations. This implies that the running time of a randomized algorithm on a given input is not fixed, but is itself a random variable.

In a deterministic algorithm, every input x uniquely determines an execution path to the output. But let us think that the **inputs are randomly coming**. Then the execution time becomes a random variable:

- random event = the execution of the (deterministic) algorithm for a random input x,
- the associated value = the execution time $T(n)$
- probability distribution: $p_t = Pr\{T(n) = t\}$

The expected time will be the expected value of the random variable:

$$exp - time(n) = E[T(n)].$$

For the particular case of discrete values for the running time from the set $\{t_0, t_1, \cdots\}$ with probabilities $\Pr\{T(n) = t_i\} = p_i$, we get

$$exp - time(n) = \sum_i t_i \cdot p_i.$$

1.2 The Hiring Problem

Suppose you own a company and need to hire a new office assistant. You decide to work with a recruitment agency, and they will send you one person to interview each day, for a total of n candidates. You want to get the best person for the job, so you decide to hire the current candidate whenever they are better than the current person on the job (whom you will fire). There are costs associated to hiring someone, and you also need to pay the recruitment agency a hiring fee. You wish to have an estimate of the cost associated with your strategy. The HireAssistant procedure describes your strategy for hiring the new office assistant.

Algorithm 1 HireAssistant

```
procedure HireAssistant(n)
    best ← 0
    for i ← 1 to n do
        Interview candidate i
        if candidate i is better than best then
            best ← i
            Hire candidate i
```

Let's assume the cost of interviewing a candidate is c_i, and the cost of hiring someone is c_h, where c_i is relatively low, and c_h is high. Then, the total cost will be $O(c_i * n + c_h * m)$, where m is the number of hired people during the process. You will interview all candidates anyways, so you are interested in analyzing the hiring cost, $c_h * m$.

In the **worst case**, the agency will send the candidates in strictly increasing order of their abilities, so you will hire n times. But in a typical situation this will not happen, so we can assume that the candidates come in random order, so that their ranks according to your hiring standards form a **uniform random permutation**. That means each of the $n!$ possible permutations of candidates appears with equal probability.

Let X be the random variable denoting how many times you hire a new assistant. Its expected value is

$$E[X] = \sum_{x=1}^{n} x \cdot \Pr\{X = x\},$$

but the calculation is cumbersome, so we will use indicator variables. First, let's recall that a random indicator variable associated with an event A indicates whether the event occurs or not: $I\{A\} = \begin{cases} 1, & \text{if event A takes place,} \\ 0, & \text{if event A does not occur.} \end{cases}$

Lemma

Given a sample space S and an event A in the sample space, let $X_A = I\{A\}$. Then $E[X_A] = \Pr\{A\}$.

Proof

By definition we have

$$\begin{aligned} E[X_A] &= E[I\{A\}] \\ &= 1 * \Pr\{A\} + 0 * \Pr\{A\} \\ &= \Pr\{A\}. \end{aligned}$$

Going back to our problem,let X_i be the random indicator variable associated to the event of hiring candidate i:

$$X_i = \begin{cases} 1, & \text{if candidate } i \text{ is hired} \\ 0, & \text{if candidate } i \text{ is not hired.} \end{cases}$$

.

We have $X = X_1 + X_2 + \cdots + X_n$, and $E[X_i] = \Pr\{\text{candidate } i \text{ is hired}\}$.

Candidate i is hired (line 6 of algorithm 1) if she is better than all candidates $1..i-1$. We assumed that the candidates come in random order, so any of the first i candidates is equally likely to be the best candidate so far. Thus, candidate i has probability $1/i$ of being the best so far and being hired, meaning $E[X_i] = 1/i$.

Now we can estimate the average cost incurred by hiring candidates.

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^{n} X_i\right] \\ &= \sum_{i=1}^{n} E[X_i] \\ &= \sum_{i=1}^{n} \frac{1}{i} \\ &= H_n. \end{aligned}$$

As the harmonic series H_n is asymptotically close to $\ln(n)$, we get that we only hire approximately $\ln(n)$ of the candidates, so an average-case total hiring cost of $c_h * \ln(n)$.

1.3 Streaks

Suppose you flip a fair coin n times. What is the longest streak of consecutive heads that you expect to see?

The probability that each coin flip is a head is 1/2. Let A_{ik} be the event that a streak of heads of length at least k begins with the i-th coin flip. More precisely, all cooin flips $i, i+1, \cdots, i+k-1$ yield only heads, where $1 \leq k \leq n$ and $1 \leq i \leq n-k+1$.

The coin flips are mutually independent, so for any given event A_{ik}, the probability that all k flips are heads is $\Pr\{A_{ik}\} = 1/2^k$.

Let $X_{ik} = I\{A_{ik}\}$ be the indicator random variable associated with a streak of heads of length at least k that starts at the $i-th$ coin flip. To count the total number of such streaks, we define

$$X = \sum_{i=1}^{n-k+1} X_{ik},$$

so its expected value will be

$$\begin{aligned} E[X] &= E\left[\sum_{i=1}^{n-k+1} X_{ik}\right] \\ &= \sum_{i=1}^{n-k+1} E[X_{ik}] \\ &= \sum_{i=1}^{n-k+1} \Pr\{A_{ik}\} \\ &= \sum_{i=1}^{n-k+1} \frac{1}{2^k} \\ &= \frac{n-k+1}{2^k}. \end{aligned}$$

By plugging in various values for k, we can calculate the expected number of streaks of length k. If this number is large (much greater than 1), then we expect many streaks of length k to occur, and the probability that one occurs is high. If this number is small (much less than 1), then we expect few streaks of length k to occur and the probability that one occurs is low. If $k = c\log n$, for some positive constant c, we obtain

$$\begin{aligned} E[X] &= \frac{n - c\log n + 1}{2^{c\log n}} \\ &= \frac{n - c\log n + 1}{n^c} \\ &= \frac{1}{n^{c-1}} - \frac{(c\log n - 1)/n}{n^{c-1}} \end{aligned}$$

so the expected value will be $O(1/n^{c-1})$.

If c is large, the expected number of streaks of length $c\log n$ is small, and we conclude that they are unlikely to occur. On the other hand, if $c = 1/2$, then we obtain $E[X] = O(1/n^{1/2-1}) = O(n^{1/2})$, and we expect that there are a large number of streaks of length $\log n/2$. Therefore, one streak of such a length is likely to occur. We can conclude from these rough estimates that the expected length of the longest streak is $O(\log n)$.

2 First Occurrence

Suppose we are given as input an array of integer numbers, a, and a target number k. We need to return the index of the first occurrence of k in a, if k appears in the array, or -1 if k is not

found in a.

We can iterate through the array one element at a time, until either k is found, and we return its index, or there are no elements left, and we return -1. The number of comparisons for the worst-case is $n = a.size()$.

Algorithm 2 First Occurrence Algorithm

```
procedure FOAlg(a, k)
    i ← 0
    while (a[i] ≠ k) and (i < a.size() − 1) do
        i ← i + 1
    if a[i] = k then
        return i
    else
        return −1
```

To compute the running time, we will look at the comparisons involving the elements of the array in the while loop.

Let's assume that the probability that $k \in \{a_0, \cdots, a_{n-1}\}$ is q, and the probability that the first occurrence of k is on position i is $\frac{q}{n}$, and let $T(n)$ denote the running time for inputs of size n. We have:

$$\Pr\{k \notin \{a_0, \cdots, a_{n-1}\}\} = 1 - q$$
$$\Pr\{T(n) = i\} = \frac{q}{n} = p_i, \forall 2 \leq i \leq n$$
$$\Pr\{T(n) = n+1\} = \frac{q}{n} + (1-q) = p_{n+1}.$$

The expected time is

$$\begin{aligned} exp - time(n) &= \sum_{i=2}^{n+1} x_i \cdot p_i \\ &= \sum_{i=2}^{n} \frac{q}{n} \cdot i + \left(\frac{q}{n} + (1-q)\right) \cdot (n+1) \\ &= n + 1 - \frac{n \cdot q}{2} + \frac{q}{2} \end{aligned}$$

For $q = \frac{1}{2}$, we get $exp - time(n) = \frac{3n}{4} + \frac{5}{4}$.
For $q = 1$, we get $exp - time(n) = \frac{n}{2} + \frac{3}{2}$.

3 Nuts and Bolts

The following problem was first proposed by Gregory Rawlins around 1992, see [2]. Suppose we are given n nuts and n bolts of different sizes to sort. Each nut matches exactly one bolt and vice versa. The sorting is done in the dark, so we can't tell if one bolt is bigger than the other, or if one nut is bigger than the other. We can only compare a nut to a bolt, by attempting to screw one into the other. The nut will be either too big, too small, or just right for the bolt. How many fitting operations do we need to match each nut to the corresponding bolt in the worst case?

3.1 Finding the nut for one bolt

Let us first consider something simpler. Suppose we want to find the nut that matches a particular bolt. The obvious algorithm — test every nut until we find a match — requires exactly $n-1$ tests in the worst case. We might have to check every nut except one; if we get to the last nut without finding a match, we know that the last nut is the one we're looking for.

Now let's use the same algorithm as before, but at each step we choose a nut uniformly at random from the untested nuts. So if there are k nuts left to test, each one will be chosen with probability $1/k$. Intuitively, the expected number of comparisons we have to perform should be about n/2, but let's formalize our intuition. Let $C(n)$ denote the number of comparisons our algorithm uses to find a match for a single bolt out of n nuts.

Possible values for $C(n) : 1, 2, \cdots, n-1$.

Base cases: $C(1) = 0$ and $C(2) = 1$.

When $n > 2$, $C(n)$ is a random variable whose value depends on our algorithm's random choices. We are interested in the expected value of $C(n)$:

$$E[C(n)] = \sum_{k=1}^{n-1} k \cdot Pr\{C(n) = k\}.$$

If the target nut is the k-th nut tested, our algorithm performs $min\{k, n-1\}$ comparisons. If the target nut is the last nut chosen, we don't actually test it, we know it is a match from the $n-1$ mismatches we tested. Because we choose the next nut to test uniformly at random, the target nut is equally likely to be the first, second, third, or k-th bolt tested, for any k (each with probability $1/n$). We make $n-1$ comparisons if the matching nut is the $n-1$-st tested, or the n-th nut that we don't test (so probability 2/n). Thus:

$$Pr\{C(n) = k\} = \begin{cases} \frac{1}{n}, k < n-1, \\ \frac{2}{n}, k = n-1. \end{cases}$$

We get the expected number of comparisons to be:

$$\begin{aligned} E[C(n)] &= \sum_{k=1}^{n-1} k \cdot Pr\{C(n) = k\} \\ &= \sum_{k=1}^{n-2} k \cdot \frac{1}{n} + (n-1)\frac{2}{n} \\ &= \sum_{k=1}^{n} \frac{k}{n} - \frac{1}{n} \\ &= \frac{1}{n}\sum_{k=1}^{n} k - \frac{1}{n} \\ &= \frac{n(n+1)}{2n} - \frac{1}{n} \\ &= \frac{n+1}{2} - \frac{1}{n} \end{aligned}$$

An alternative way of getting to the same answer is by thinking of this algorithm recursively. We always have to perform at least one test. With probability $1/n$, we successfully find the

matching nut and halt. With the remaining probability $1 - 1/n$, we recursively solve the same problem but with one fewer nut. We get the following recurrence:

$$\begin{cases} C(1) = 0, \\ E[C(n)] = 1 + \frac{n-1}{n} E[C(n-1)], \text{ for } n > 1. \end{cases}$$

Let's define the function $c(n) = nE[C(n)]$, and rewrite the previous recursion:

$$\begin{cases} c(1) = 0, \\ c(n) = n + c(n-1). \end{cases}$$

This translates to a summation, $c(n) = \sum_{k=2}^{n} k = \frac{n(n+1)}{2} - 1$. Going back to $E[C(n)]$, we get

$$E[C(n)] = \frac{c(n)}{n} = \frac{n+1}{2} - \frac{1}{n}.$$

3.2 Matching all bolts and nuts

Let us go back to the initial problem, of matching each nut to its corresponding bolt. The simplest way to solve the problem would be to compare each nut to each bolt, for a total of n^2 comparisons. Next, we may try to arbitrarily find a matched pair, using the algorithm in the previous section, for a total number of comparisons of

$$\sum_{k=1}^{n} (k-1) = \frac{n(n-1)}{2}.$$

Let's consider something better, proposed by Rawlins when he introduced the problem. We chose a pivot bolt that we test against every nut, to sort the unmatching nuts in two piles: smaller than the pivot bolt, respectively larger than it. Then test the matching nut against every other bolt, to sort the bolts into two piles, compared to the pivot bolt. After these $2n-1$ comparisons, we get a matching bolt-nut pair, and two smaller size subproblems of our initial problem that we need to recursively solve.

3.2.1 Worst-case analysis

The worst-case number of comparisons made by this algorithm is given by the recurrence

$$\begin{cases} C(0) = 0 \\ C(n) = 2n - 1 + \max_{1 \le k \le n} \{C(k-1) + C(n-k)\} = 2n - 1 + C(n-1). \end{cases}$$

The recurrence solves to

$$\begin{aligned} C(n) &= \sum_{i=1}^{n} (2i-1) \\ &= 2\sum_{i=1}^{n} i - n \\ &= 2\frac{n(n+1)}{2} - n \end{aligned}$$

Algorithm 3 Match Nuts and Bolts

```
procedure MatchNutsAndBolts(N, B, low, high)
    if low < high then
        p ← ChoosePivotBolt(B, low, high)                          ▷ Choose a pivot bolt
        pivotNutIndex ← Partition(N, low, high, B[p])              ▷ Partition nuts using pivot bolt
        pivotBoltIndex ← Partition(B, low, high, N[pivotNutIndex])  ▷ Partition bolts using matching nut
        MatchNutsAndBolts(N, B, low, pivotNutIndex − 1)            ▷ Solve left subproblem
        MatchNutsAndBolts(N, B, pivotNutIndex + 1, high)           ▷ Solve right subproblem

function ChoosePivotBolt(B, low, high)
    return random index from [low, high]

function Partition(A, low, high, pivot)
    i ← low
    for j ← low to high − 1 do
        if Compare(A[j], pivot) = −1 then                          ▷ Element smaller than pivot
            Swap(A[i], A[j])
            i ← i + 1
        else if Compare(A[j], pivot) = 0 then                      ▷ Move pivot to end
            Swap(A[j], A[high])
            j ← j − 1
    Swap(A[i], A[high])                                            ▷ Move pivot to correct position
    return i

function Compare(x, y)
    if x < y then
        return −1
    else if x > y then
        return 1
    else
        return 0
```

$$= n^2$$

In the worst-case, this algorithm tests every nut-bolt pair. However, since the recursion looks a lot like quicksort, and the average-case running time of quicksort is $\Theta(n\log n)$, it seems reasonable to guess that the average number of nut-bolt comparisons is also $\Theta(n\log n)$. We shall see that if the pivot bolt is always chosen uniformly at random, this intuition is correct.

3.2.2 Expected number of comparisons (average-case)

Assume our algorithm chooses its pivot bolt uniformly at random from the set of unmatched bolts. Then the selected pivot bolt is equally likely to be the smallest, second smallest, or k-th smallest for any k, i.e. a bolt appears on position k in the sorted list with probability $1/n$.

Analysis Using Indicator Variables

Let's consider an iterative version of our algorithm that does not change the number of comparisons:

1. Choose a pivot bolt, find its match, and partition the remaining nuts and bolts into smaller-than and larger-than the matching pair.
2. Pick the second pivot bolt, find its match and partition the subset that contains it. We get two matching pairs and three subsets.
3. Continue choosing new pivot bolts and partitioning the subsets containing them, each time finding one match and increasing the number of subsets by one, until every bolt has been chosen as the pivot.
4. At the end, as all bolts were pivots, they have their matching nuts.

Suppose we always choose the pivot bolt uniformly at random from the bolts that haven't yet been pivots. Then any bolt is equally likely to be selected as the pivot bolt, no matter the subset that contains it. It follows that this randomized iterative algorithm performs exactly the same set of tests as the randomized recursive algorithm, but possibly in a different order.

Let B_i denote the i-th smallest bolt, and N_j denote the j-th smallest nut. For each i and j, we define an indicator variable

$$X_{ij} = \begin{cases} 1, & \text{if } B_i \text{ and } N_j \text{ are compared} \\ 0, & \text{otherwise} \end{cases}$$

Then the total number of nut-bolt comparisons is exactly

$$C(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} X_{ij}.$$

The expected number of comparisons is

$$E[C(n)] = E\left[\sum_{i=1}^{n} \sum_{j=1}^{n} X_{ij}\right] = \sum_{i=1}^{n} \sum_{j=1}^{n} E[X_{ij}]$$

First let's assume that i < j. Our algorithm performs comparisons between some pivot bolt and a nut in the same subset (or between the pivot's matching nut and the bolts in the same

subset). The only event that can prevent a comparison between B_i and N_j is choosing some intermediate pivot bolt B_k, with $i < k < j$, before B_i or B_j. In other words, B_i and N_j are compared iff the first pivot from the subset $B_i, B_{i+1}, \cdots, B_j$ is B_i or B_j. Since the subset $\{B_i, B_{i+1}, \cdots, B_j\}$ contains $j - i + 1$ bolts, and each bolt is equally likely to be chosen first, we have

$$E[X_{ij}] = \frac{2}{j - i + 1}, \forall i < j.$$

Similarly, $E[X_{ij}] = \frac{2}{i-j+1}, \forall i > j$. For equal indices we have $E[X_{ii}] = 1$. Putting everything together, we get

$$\begin{aligned}
E[C(n)] &= \sum_{i=1}^{n} \sum_{j=1}^{n} E[X_{ij}] \\
&= \sum_{i=1}^{n} E[X_{ii}] + 2 \sum_{i=1}^{n} \sum_{j=i+1}^{n} E[X_{ij}] \\
&= n + 4 \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{1}{j - i + 1} \\
&= n + 4 \sum_{i=1}^{n} \sum_{k=2}^{n-i+1} \frac{1}{k} && \text{(substitute } k = j - i + 1\text{)} \\
&= n + 4 \sum_{k=2}^{n} \sum_{i=1}^{n-k+1} \frac{1}{k} && \text{(reorder summations)} \\
&= n + 4 \sum_{k=2}^{n} \frac{n - k + 1}{k} \\
&= n + 4 \left((n+1) \sum_{k=2}^{n} \frac{1}{k} - \sum_{k=2}^{n} 1 \right) \\
&= n + 4((n+1)(H_n - 1) - (n-1)) \\
&= n + 4(nH_n - 2n + H_n) \\
&= 4nH_n + 4H_n - 7n.
\end{aligned}$$

Given that $H_n = O(\ln(n))$, we get the expected number of comparisons $O(n\ln(n))$.

Recursive Analysis

Let Y_k be the random variable that returns the expected number of comparisons for the recursive calls if the pivot is the k-th element, 0 otherwise:

$$Y_k = \begin{cases} E[C(k-1)] + E[C(n-k)], & \text{if the pivot is } B_k, \\ 0, & \text{otherwise} \end{cases}$$

The expected value of Y_k is

$$E[Y_k] = \frac{1}{n} \left(E[C(k-1)] + E[C(n-k)] \right)$$

The total number of comparisons is $C(n) = 2n - 1 + \sum_{k=1}^{n} Y_k$ so the expected number of comparisons performed by our algorithm is

$$\begin{aligned}
E[C(n)] &= E[2n - 1 + \sum_{k=1}^{n} Y_k] \\
&= 2n - 1 + \sum_{k=1}^{n} E[Y_k] \\
&= 2n - 1 + \frac{1}{n} \sum_{k=1}^{n} (E[C(k-1)] + E[C(n-k)]) \\
&= 2n - 1 + \frac{1}{n} \left(\sum_{k=0}^{n-1} E[C(k)] + \sum_{k=0}^{n-1} E[C(k)] \right) \\
&= 2n - 1 + \frac{2}{n} \sum_{k=0}^{n-1} E[C(k)]
\end{aligned}$$

Multiplying by n, we get

$$nE[C(n)] = n(2n-1) + 2\sum_{k=0}^{n-1} E[C(k)].$$

Similarly,

$$\begin{aligned}
(n-1)E[C(n-1)] &= (n-1)(2n-3) + 2\sum_{k=0}^{n-2} E[C(k)] \\
&= 2n^2 - 5n + 3 + 2\sum_{k=0}^{n-2} E[C(k)].
\end{aligned}$$

We subtract the two previous equations:

$$\begin{aligned}
nE[C(n)] - (n-1)E[C(n-1)] &= 2n^2 - n + 2\sum_{k=0}^{n-1} E[C(k)] - 2n^2 + 5n - 3 - 2\sum_{k=0}^{n-2} E[C(k)] \\
&= 4n - 3 + 2E[C(n-1)] =>
\end{aligned}$$

$$\begin{aligned}
nE[C(n)] &= 4n - 3 + (n+1)E[C(n-1)] => \\
E[C(n)] &= \frac{n+1}{n} E[C(n-1)] + 4 - \frac{3}{n}
\end{aligned}$$

Let $t(n) = \frac{E[C(n)]}{n+1}$. We get

$$\begin{aligned}
t(n) &= \frac{1}{n+1} \left[(n+1)t(n-1) + 4 - \frac{3}{n} \right] \\
&= t(n-1) + \frac{4}{n+1} - \frac{3}{n(n+1)} \qquad \left[\frac{1}{n(n+1)} = \frac{1}{n} - \frac{1}{n+1} \right]
\end{aligned}$$

$$= t(n-1) + \frac{7}{n+1} - \frac{3}{n}$$

The base case is $t(0) = 0$. The recurrence directly translates to a summation:

$$\begin{aligned} t(n) &= \sum_{i=1}^{n} \left(\frac{7}{i+1} - \frac{3}{i} \right) \\ &= 7 \sum_{i=1}^{n} \frac{1}{i+1} - 3 \sum_{i=1}^{n} \frac{1}{i} \\ &= 7(H_{n+1} - 1) - 3H_n && \left[H_{n+1} = H_n + \frac{1}{n+1} \right] \\ &= 4H_n - 7 + \frac{7}{n+1} \end{aligned}$$

We multiply by $n+1$ and go back to $E[C(n)]$:

$$E[C(n)] = 4nH_n + 4H_n - 7n,$$

which is exactly what we got in our first analysis.

3.2.3 Reduction to and from sorting

Rawlins' algorithm for matching the nuts and bolts strongly resembles quicksort. The algorithm not only matches up the nuts and bolts, but also sorts them by size.

Remember from our previous lectures that (polynomially) reducing a problem P to a problem Q means solving P by using an algorithm that solves Q, with some polynomial pre-processing of the input data and post-processing of the output. Can you think of a reduction from Nuts and Bolts to Sorting, or in the other direction?

Reducing Sorting to Nuts and Bolts is quite straightforward: duplicate the array to be sorted (one copy will function as bolts, the other as nuts), call the algorithm for matching nuts and bolts, and by construction of that algorithm, the nuts and bolts will not only match, but be sorted, so the initial array will be sorted. No post-processing is needed.

We could reduce Nuts and Bolts to Sorting. If the bolts were sorted (with a known $O(n\log n)$ algorithm), we could match the nuts and bolts in $O(n\log n)$ time by performing a binary search with each nut. Thus, if we had an algorithm to sort the bolts, we would immediately have an algorithm to match the nuts and bolts, starting from scratch, in $O(n\log n)$ time. Or, if we were able to compare nuts with nuts, and bolts with bolts, we could independently sort the nuts and the bolts respectively, and the pairings would be between nuts and bolts of the same index in the sorted arrays.

Unfortunately, since we aren't allowed to directly compare two bolts or two nuts, we can't use a sorting algorithm directly to sort the nuts and bolts in $O(n\log n)$ worst case time. The problem of sorting nuts and bolts deterministically in $O(n\log n)$ time was solved in 1995, see [1].

The reductions show us that, if we have an $O(n\log n)$ time algorithm for either sorting or matching nuts and bolts, we automatically have an $O(n\log n)$ time algorithm for the other problem.

References

[1] Phillip G Bradford. Matching nuts and bolts optimally. 1995.

[2] Gregory JE Rawlins. *Compared to what? An introduction to the analysis of algorithms.* Computer Science Press, Inc., 1992.